数据结构 · 顺序表算法题 · 考点精讲与真题整合

408 各科大题知识点强化课 | 课程第 1 讲（分 P2，全课程开篇） | 时长 3 小时 21 分 25 秒

本讲范围：这是全课程的第 1 讲，一共干了两件事。前 40 分钟**不讲题**，讲考情与复习规划：408 分值结构、数据结构大题的分布（哪几小节出过大题、各出过几道）、算法题的定位（只占 15 分）、以及他给这门课定的复习节奏。之后进入本讲正题——**数据结构第二章 2.2 顺序表的算法大题**，他完整走了**五道真题（书上第 10～14 题，即 2.2 节那 5 道代码题）**，另外补了两道非真题（第 2 题数组反转、第 6 题两有序序列归并），以及三个高频算法思想（双指针、辅助数组当哈希表、C++ `map`）。

读法建议：本讲的题本身不难，他反复说「题目本身不难」「通过这个题目了解一下真题里面算法题是什么样的形式去考」。真正该背的是 **§3 的作答规范与三条开数组纪律**和 **§4 里五道题各自的「通用解」**——他一再强调**不要看答案的最优解**（「换个题目就没法用了……因为答案给的都是最优解」），要看他课上写的通用做法。

一句话先记住：算法题占 15 分，**写成暴力解 / 通用解、三问（思路 + 代码 + 复杂度）都答满，保底能拿 11 分左右**；空着才是真的 0 分。

目录

1 考情分析：分值、大题分布、复习节奏

2 核心概念：算法题怎么考、怎么评分

3 解题方法论：作答规范与三条开数组纪律

4 典型题型与模板：五道真题 + 两道非真题

5 真题年份对照与认定依据

6 易错点清单

7 考场速查卡

附 本资料的来源与边界

ASR 校正说明（读引用前先看这一段）

讲稿由本地 ASR 转写，**这一讲的错字密度比后续各讲都高**（老师第一次上课、语速偏快、简繁混杂），而且大量错字集中在**变量名和代码术语**上。引用时已统一校正，最要紧的几条是：

① **变量名（本讲最容易读错的一类）**：`爱` → `i`；`这` → `j`；`配` → `k`；`框 / 矿 / quant / compt / comment` → `cnt`（计数器）；`A&S / nns / NS / ans` → `ans`；`Time / temp / tamp / 叹夫 / 碳粉` → `temp`（辅助数组名）；`N` 与 `L` 在讲中位数那题时转写会混。凡把「碳粉爱」照抄进代码的地方，实为 `temp[i]`。

② **术语**：`大体 / 大帖 / 大铁 / 整体 / 真体 / 争批` → 大题 / 真题；`速度 / 宿主 / 输出 / 副组 / 处处 / 数卒 / 元素组` → 数组；`顺域表 / 顺序表`、`练表 / 列表` → 链表；`上法题 / 藏法题` → 算法题；`与法 / 愈法 / 余法` → 语法；`辅达度 / 复达度 / 拍箱` → 复杂度；`追优解 / 追优节 / 最右解 / 自由解` → 最优解；`组元数 / 组元素 / 主员数` → 主元素；`中卫术 / 中维数 / 综卫处` → 中位数；`规定 / 规定排序` → 归并 / 归并排序；`模耳投票` → 摩尔投票；`输走` → 枢轴；`意图性 / 异毒性` → 易读性；`王道说 / 舞蹈 / 王道叔` → 王道书；`控制 / 控的` → 空着；`健康老师 / 感恩老师 / 改建老师` → 判卷老师；`哈西表 / 哈奇表` → 哈希表；`临阶矩阵` → 邻接矩阵；`最小生存数` → 最小生成树；`技术排序` → 计数排序；`散裂表` → 散列表；`合办查找` → 折半查找；`二插数 / 二叉数` → 二叉树；`锁碎` → 琐碎。

③ **代码写法**：`maloc / 麦乐卡 / Milok` → `malloc`；`SizeO` → `sizeof`；`6 英特` → `new int`；`int-max` → `INT_MAX`；`eat-first / eat-second / 杠箭头` → `it->first / it->second / ->`；`加碼` → Java；`零 X 表示 16 禁止` → `0x` 表示十六进制；`草稿纸`（转写作「超高纸」）。

④ **两处必须单独说明**：`声语员 / 声加加` → **C 语言 / C++**；`开始 / 快表` 在讲「哪些知识点也考选择题」时指 **cache 与快表**（依据是他紧接着说「学到（计）组 cache 和指令很多大题」，见 `[00:20:57]`）。

⑤ **一处本资料拒绝猜测**：他在数非代码题分布时有一句转写作「最烈考了一道」（`[00:19:28]`），上下文与王道书章节都对不上，**本资料不代为认定是哪个知识点**，见 §附⑤。

1 考情分析：分值、大题分布、复习节奏

1.1 分值结构与时间定位（开场第一件事）

> 「因为现在已经 9 月份了……现在已经 9 月 5 号了，所以就是说 408 的话要尽快开始准备大题。……408 是 **40 个选择题，每个选择题 2 分**，然后是 **80 分**；然后大题是 **7 道**，7 道的话总共是 **70 分**。所以 408 的大题叫**综合应用题**，就是它的大题的综合性要更高点。」（`[00:10:02]` – `[00:10:56]`）

> 「尤其这些大题，408 里一些大题第一遍做的话可能会不太好做，甚至说做了两遍可能还是不太好做。一般来说可能要做到**第三遍**左右，可能真正完全把它搞透。」（`[00:11:06]` – `[00:11:19]`）

他给这门课定的四条调子（背这四句就够）

① **大题占了 70 分，但别忽视选择题**：「你可以观察一下，**凡是能考大题的一定能考选择题**。所以其实大题占了 70 分，然后大题相关的这些知识点占的分是**绝对不止 70，应该要更多，可能有这个 90 多分**……比如这个数据结构的最短路径、最小生成树那些算法，cache、快表这些，他大题考、选择题也考。」（`[00:11:31]`–`[00:12:12]`）

② **算法题只占 15 分，别为它拼命**：「因为 408 这个 150 分，**算法题的话只占 15 分**，剩下的其实都和代码没关系。所以这个 15 分你要完全放弃的话，这个还是比较可惜的；但是说你为了这 15 分花费特别多的精力，也是不合理的。」（`[00:12:37]`–`[00:12:57]`）

③ **每个类型做四五道就够**：「你这 15 分基本就是你把各个类型——像能考代码的就是**数组、链表、二叉树、图**，每一个类型肯定都要做的，然后**每一个类型都做个四五道题**都了解一下就够了。」（`[00:12:57]`–`[00:13:13]`）

④ **代码只许用 C / C++**：「代码的话一般是要用 **C 和 C++** 写的，**其他语言一般是不允许的**」（`[00:13:31]`–`[00:13:36]`）；他后面又补了一句「**Java 现在是不让用了，从 15、16 年开始就不让用了**」（`[00:42:41]`–`[00:42:45]`）。

1.2 数据结构大题的分布（他让「截一下图」的那两张）

「这个数据结构**每年是两道大题**：就**一道代码题，就是算法题**，另外一道就是**非算法**——各出一道。」（`[00:14:52]`–`[00:15:07]`，转写作「各处一道」）

「408 大题有一个特点，考得比较集中……你会发现王道书上有的知识点它是只有选择，大题往往固定在个别的几个小节。」（`[00:15:13]`–`[00:15:28]`）

代码题（算法题）的章节分布——共 15 道（他给的题号是王道书小节号）：

王道小节	题量	他的话
2.2 顺序表	**5 道**	「在书上的 2.2，就是顺序表……这边是有 5 道」（`[00:16:45]`–`[00:16:55]`）← **本讲内容**
2.3 链表	**4 道**	「然后 2.3 链表，这边是有 4 道大题」（`[00:16:55]`–`[00:17:07]`）← 下一讲内容
5.3 二叉树的遍历	**3 道**	「然后比如说 5.3，二叉树的遍历……二叉树这边还有三道」（`[00:17:13]`–`[00:17:42]`）
图的存储	**2 道**	「图的存储的话有两道，有和图的邻接矩阵、邻接表这些相关的两道」（`[00:17:45]`–`[00:18:02]`）
快排	**1 道**	「快排，还有一道代码题」（`[00:18:04]`–`[00:18:06]`）
合计	**15 道**	「然后总共是 15 道，刚好就 15 道。」（`[00:18:15]`–`[00:18:24]`）

他还交代了王道书的版本边界：「他这个应该是从 **09 年到 23 年**的真题都有的，**你王道书上是不包含 24 年真题的**」（`[00:16:15]`–`[00:16:23]`）。紧接着一句转写作「09 到那是 15 年的整体」，⚠ 转写含糊，但按上下文与总数可以判定他说的是**「09 到 23 是 15 年的真题」**——与「每年两道大题」正好推出 15 道代码题、15 道非代码题，两处数字自洽（见 §附③）。

非代码题的分布——他说「也总共是 15 道」（他只报了小节名与数量，没有展开讲）：队列 1 道（「比较低频」）、二叉树的节点计算 1 道、哈夫曼树 2 道、图的存储 1 道、图的应用 **5 道**（含最小生成树 Prim / Kruskal、最短路径 Dijkstra、关键路径）、折半查找、散列表（「让你算成功的平均查找长度和失败的平均查找长度」）、堆排序 1 道、计数排序、外部排序（`[00:19:19]`–`[00:22:13]`）。⚠ 这一串里有一句转写作「最烈考了一道」，含义不明，未计入上表。

> 「其实也就数据结构的大题会特别分散。后面学到（计）组、你像操作系统 PV 操作和分页很多大题，学到（计）组 cache 和指令很多大题，然后学到计网 IP、TCP 很多大题，那些其实特别集中；**数据结构这门课它其实有点琐碎**，所以就是这个大题的分布会比较分散。」（`[00:20:49]`–`[00:21:11]`）

1.3 为什么这一讲先讲顺序表

> 「前期刚学代码题最好是先学**顺序表、链表**，因为它的代码会简单一点……数（据结构）图的代码是不太适合入门上来就学的。」（`[00:17:26]`–`[00:19:04]`）

> 「（顺序表这边）上次还是 2020 年考的，就挺长时间没考了。因为前几年连续考了好几年的数和图——**去年考的图，然后前年考的数，再前年考的图**。这几年数图考得很多，所以顺序表、链表挺久没有考了，**一定要重视一下**。但为了考试，最好还是说复习得全面一点，就顺序表、链表、数图都要会一点……**他无论考到什么你都能拿一点分的**。」（`[00:57:01]`–`[00:57:44]`，⚠ 转写作「就挺堂之間沒考了」「但是速度不会要比这个强」，按上下文校正为「挺长时间」「数图」；他说「去年 / 前年 / 再前年」，**没有点明年份**）

1.4 他给的时间表

时间	任务（他的话）
现在 ~ 10 月中旬	「把四本书的大题学完」（`[00:16:33]`）；「这个（专题）课的安排一定要在 10 月中旬把这个大题做**两到三遍**——课堂上讲一遍，课下再亲自动手做个两遍」（`[00:35:08]`–`[00:35:21]`）
10 月中旬	「然后 10 月中旬的话就可以做这个**真题卷**了」（`[00:16:37]`）
算法题 / 选择题	算法题「讲完之后**一定要写四到五遍**……今天刚讲你可以明天写一遍，然后隔个三四天写一遍，这样重复个两三遍，之后再隔个半个月再去写两遍」（`[00:13:43]`、`[03:16:31]`）；选择题「有时间还是要把四本书非算法题、选择题那些概念再好好复习复习」，「**争取考前过个三四遍**」（`[03:17:07]`–`[03:17:30]`）

> 「408 学到现在应该都能感觉到，这个主要的问题就是**容易忘**。东西倒是不难，最大的问题就是那些东西学完容易忘，**这个代码是更容易忘的**……所以就是一定要写四五遍。然后**不要硬背，代码不推荐硬背**：你写的遍数多了，你自然一些语法就都熟了」（`[00:13:52]`–`[00:14:25]`）。

2 核心概念：算法题怎么考、怎么评分

2.1 三段式：每一问分别考什么

> **本讲最该背的一条：算法大题 = 三段式**
>
> 「一般考察的话是分为**三段式**：**第一小问会问你这个思路**，让你写思路——**中文写思路都是有分的**，就考试的话，哪怕这个算法题不会写代码，有思路都可以写上去，**一定不要空着**；**第一问思路，第二问一般是代码实现**，然后**第三问一般是让你分析时空复杂度**——分析你自己的代码时空复杂度。」（`[00:25:10]`–`[00:25:43]`）
>
> 分数结构：「如果你的时空复杂度高一点，那可能分值就会低一点；但是**一定不要空着**，哪怕你采用的是暴力解。」（`[00:25:46]`–`[00:26:05]`）

2.2 最优解 vs 暴力解 / 通用解（本讲的中心论点）

	最优解	暴力解 / 通用解
他的原话	「这个最优解**往往不太好想到**」（`[00:26:42]`）；「换个题目就没法用了……因为答案给的都是最优解」（`[03:07:16]`–`[03:07:19]`）	「它的开销比较大，可能多写一些循环、多开一些辅助数组……**但是它一般这个代码会比较好实现，而且思路比较容易想到**」（`[00:26:23]`–`[00:26:42]`）
得分	—	「你即便用了暴力解，满分 15 分……**他也会给一个 11 到 12 分，扣的不会太多**」（`[00:26:48]`–`[00:26:58]`）
要不要学	不建议：「像这种思路其实**没必要学的，学了没有太大意义**，你换个题目这种思路没啥用的」（`[00:37:12]`–`[00:37:18]`）	**练习重点**：「所以我们练习的重点肯定是这个暴力解和通用解」（`[00:26:44]`）

> **反过来说：只写思路不写代码，分很少**
>
> 「如果满分 15 分，你最后代码什么解法都没写、空着了，只写一个思路，那可能给的分就很少，**一定要完整**——只要有思路、有代码、有（时空）复杂度，哪怕开销大，分值也都是比较高的。他不会因为（不是）最优解扣你太多的分数，一些小的语法错误也不会扣太多。」（`[00:27:02]`–`[00:27:24]`）

王道书的局限：「王道书他是**只给了最优解**，别的解法只提供了一个思路、没有给代码，所以我们课堂上会讲一些和答案不同的那个代码」（`[00:30:52]`）。

2.3 三个高频算法思想（他说「都是特别重要、特别高频的」）

> 「我们上课今天讲的算法（思想）都是特别重要的、都是特别重要特别高频的，那种很冷门的都是没讲的。」（`[01:52:58]`–`[01:53:03]`）

他把双指针拆成三类，并在本讲就先把数组侧的用法练完（链表侧的留到下一讲）：

类别	用途（他的话）	出处
左右指针	「第一个叫**左右指针**，它的用途的话就是**数组反转、快速排序**」	`[01:02:47]`–`[01:03:13]`
快慢指针	「还有一个叫**快慢指针**……（链表倒数第 k 个节点、链表找终点、相交链表）这些都会用到快慢指针的思路」——他明确说「**讲到链表的话再去举具体的例子去展开**」	`[01:09:01]`–`[01:09:55]`
归并	「第三种的话叫**归并**，也就是刚才我们讲的这道题会用到的……**它分为数组归并和链表归并**」	`[01:09:58]`–`[01:10:26]`

> 第二个思想：辅助数组当哈希表（统计出现次数 / 是否出现）
>
> 「这个叫**数组当 map**，也是一个很高频的算法思想」（`[01:52:53]`–`[01:52:58]`）。
> 用法：「**把这个 a 数组的值当成辅助数组的下标**」（`[01:54:43]`）。他现场算了一遍：原数组的数都在 0~6 之间、最大是 6，所以辅助数组要**开 7 个**；统计完辅助数组是 `0 1 0 2 0 1`——数值 1、2、6 各出现 1 次、4 出现 2 次，下标 0、3、5 是 0，**说明 0、3、5 在原数组里没出现过**（`[01:53:42]`–`[01:59:16]`；⚠️ 转写没有念全这个例子的原始元素与顺序，本资料只保留他报出的统计结果）。
> 两个前提条件（他单独强调）：「首先原数组的数值都必须**是正整数**——其实不是说必须是正数，是必须是正整数，因为你一个小数、一个负数都不能作为下标的，当然**零也可以**；其次你必须**提前知道它的范围**，你只有提前知道原数组的数值范围，你才能去开辅助数组」（`[01:59:21]`–`[02:00:05]`）。
> 用法有两种：「有时候也会用来**统计是否出现**……那你就看下标，看它的数值是否是零就可以」（`[01:58:47]`–`[01:59:01]`）。

第三个是 C++ 自带的 `map`——他把它定位成「压箱底的拓展」，见 §4.7。

3 解题方法论：作答规范与三条开数组纪律

3.1 写代码的七条规范（全部出自他的口播）

> ① 只写一个核心函数；② 把已知条件都当成函数的参数。
> 「考试的话是只写一个核心函数」[00:23:43]；「你把已知条件都当成参数就可以了」[00:23:58]
> ③ 头文件、main 函数、输入输出都不要写。
> 「头文件，就是 #include 的那些东西都不要写的」「和 main 函数（转写作「闷函数」）包括输入输出这些都不要写」[00:24:10]–[00:24:38]
> ④ 判卷老师不会运行你的代码，重点是思路正确。
> 「你的代码最终其实判卷老师也不会去运行的，你的代码只要写出来没有太大问题就行，就是思想正确、没有太多太明显的语法错误就够了，有一点点小的语法错误这些都问题不大，重点是思路正确。」[00:24:39]–[00:24:52]

⑤ 易读性要高，一定要加注释。
「你的代码一定要易读性比较高，因为考试你的代码是要交给老师看的」[00:24:54]；
「你的代码不是给自己看的，是给监考老师看的」[00:27:51]
⑥ 常见的小函数直接用，不要自己实现（swap、求绝对值、取最大、取最小）。
「如果考试中你要用到的话直接用就行了，没必要去实现了……这种东西它不会因为你多写的这个、多给你两分」[00:28:33]–[00:29:09]
⑦ 代码要短。
「答案的代码往往都不会超过 15（行）啊，往往都是 10 行以内，都很精简。把最核心的代码给它展示出来就可以了」[00:29:12]–[00:29:27]

关于注释加在哪，他在第 12 题里补了具体做法：「加注释的话一定要加在这里：你比如这个循环，或者你辅助数组干嘛用的，你肯定要写一下注释——**用来统计（原）数组每个元素出现的次数**……你加注释的目的是别人看的，不要乱加，加的一定要简洁，加在该加的位置：有一个循环、或者是关键的判断语句加一下就可以了，你不用加得太多，核心的几个地方一加就可以了」（`[02:12:35]`–`[02:13:16]`）。他还给了一条判卷侧的观察：「所以你一加上注释，监考老师就不会具体去看你的代码，就看一下你的注释就行了」（`[00:54:18]`–`[00:54:24]`）。

答题版面：先答案、后过程。「408 所有的大题一般都不太看重计算过程，**能简洁就简洁，一定要把答案放在比较显眼的位置**。因为 408 你做过一些大题的话会发现，经常一个题目有好几个小问、每个小问又有好几个小问，所以你到时候一定要先把答案放到显眼的位置，过程的话随便写一点就行了……因为他平均一个小问就一两分，然后这个过程一定不要写得太啰嗦，**越精简也越好**。」（`[01:26:00]`–`[01:26:33]`）时空复杂度「直接写结果就行，不需要分析的；当然你如果考试时间比较宽裕，其实你也可以简单说一下……最好还是稍微写，不要写太长」（`[01:25:28]`–`[01:25:57]`）。

3.2 开辅助数组的三条纪律

纪律一：动态数组要用 malloc / new，不要写 int a[p]

「有的人可能习惯性地会这样去写（`int a[p]`），**这个不太推荐**……它其实在 C++ 里面也可以，因为考试 C 语言 C++ 都行；但是这种写法一般编译器不会通过，所以要写得标准一点……你翻王道书，课后答案一般都是用 **C 语言**去实现的，**推荐用上面这个**（`malloc`），会和大多数真题的写法保持一致。」（`[00:42:17]`–`[00:45:14]`）

模板（他念的标准写法，C 与 C++ 各一行）：

```
int *temp = (int *)malloc(n * sizeof(int));  /  int *temp = new int[n];
```

纪律二：开完辅助数组一定要手动清零

「注意这个考试写代码，你最好加上这样的一些代码：就是你**每开完一个辅助数组，最好你把它（赋）一下**，因为有的地方它可能会有一些脏数据，有的内存里面可能有一些旧的数据，所以你最好手动把它（赋）一下，**把辅助数组的每一个元素先赋成零**，最好手动操作一下。其实也可以当作凑一行代码——一般来说（真）题的代码比较短，你多写一行稍微凑一下，多写这一行代码也不会变得很长。」（`[02:07:00]`–`[02:07:36]`，转写作「把它复制一下」）

> **纪律三：辅助数组的大小要「慎重考虑」，写完前后要对齐**
>
> 「就这个辅助数组到底开多大，到时候做的时候可以**慎重思考**呀，这个很关键的……都可以多开的，但是多开的话就是看你需要多大。」（`[02:46:09]`–`[02:46:27]`）
>
> 「（第 13 题）开 N 个是最麻烦的，因为那个下标要减 1 去放……你开 N+1 跟 N+2 其实都还好。N 个的话下标去减 1 这个很麻烦，相当多了一些反锁的步骤，**很容易写着写着就乱了**：你可能放的时候下标减 1 了，拿的时候又忘了加……反过来的时候又忘了加 1，很容易漏掉。」（`[02:46:34]`–`[02:46:57]`）
>
> 配套的一句话：「你要按自己的写法去写，**前后都要对应**。」（`[02:36:34]`–`[02:36:38]`）

3.3 三个可照抄的步骤模板

```
模板 A | 左右指针（数组反转、快排的划分）
 1 左指针 i = 0，右指针 j = n-1
 2 while (i < j)：交换 a[i] 与 a[j]；i++；j--      ← 相遇即停，中间那个不用管
 ★ 快排的划分是同一套指针 + 三条附加性质（见 §4.3）

模板 B | 归并（两个有序序列合成一个有序序列）
 1 两个指针 i、j 分别指向两个序列的开头，k 指向结果序列
 2 while (i < L && j < L)：谁小取谁，取完那个指针往后走
 3 循环退出后：哪个序列还有剩，就把剩下的全部接在后面
 ★ 「当一个序列走完之后，那你把剩下这个序列的全部接在后面就可以了」[01:12:21]–[01:12:29]
 ★ 为什么一定要有第 3 步：「第一个序列它的数字都很小，第二个序列的数字
   很大，很有可能第一个走到头了，第二个还刚开始走，人家要处理这种情况」[01:20:06]–[01:20:17]

模板 C | 辅助数组统计（当哈希表用）
 1 开数组：大小 = 值域上界 + 1（值最大是 6 就开 7 个）；手动清零
 2 遍历原数组：temp[a[i]]++          ←「把 a 数组的值当成辅助数组的下标」
 3 再遍历辅助数组：为 0 即没出现；或用次数判断条件
```

3.4 答题节奏上的两条提醒

> 「比如说一个大题可能有多个小问，不同小问之间可能是**递进**的，也可能是**毫无关联**。如果不同的小问是递进的，可能题目比较难，它会一步一步深入地去问；如果不同小问没关联，那前面小问如果某一个不会做，你到时候就可以**先做后面的**。」（`[00:33:19]`–`[00:33:42]`）

> 「你应该先有一个大体的思路，然后**把代码写完之后你再去写第一问**，这样的话你的思路和你的代码会比较对应。」（`[02:19:15]`–`[02:19:33]`）

> 「（写代码时）有时候下标会很容易多一个少一个。**你写代码的时候，一定要通过一个具体的例子去代一下**，看你的代码能不能正常运行。」（`[00:55:45]`–`[00:55:57]`）

4 典型题型与模板：五道真题 + 两道非真题

他这一讲的顺序是：**先拿最简单的一道真题（循环左移）走完「思路→代码→复杂度」的完整流程**，再讲两个非真题引出双指针与快排，然后用同一套指针讲中位数、主元素、最小正整数，最后收在 2020 年那道三数组最

短距离上。他解释了为什么这么排：「因为他讲了一道题讲得特别细，然后（后面）就属于（换）一下这种代码题的风格。这边这几道题它考察的点都不一样——这边五道题每个题目考察的知识点虽然可能每个题都并不算太难，但是**考察的点都不太一样**，所以这边还是要多讲一下」（`[00:58:54]`–`[00:59:15]`）。

4.1 2010 循环左移（王道 2.2 第 10 题，书上 19 页）

题面要点：一维数组 `R`，长度 `n`，**循环左移 p 个位置（0 < p < n）**。他举的例子是 `6159847`（7 个数据）左移三位 → `9847 615`（`[00:35:48]`–`[00:36:36]`）。

① 答案的最优解（三反转）——他明确不推荐。「这个答案是这样做的：你先把前三个反转变成 516，再把后面的反转 7489，接下来再把整个反转……**先把前 p 个反转、再把后面的反转、再整个反转。**」（`[00:36:44]`–`[00:37:08]`）复杂度他算得很清楚：「你把前面反转就是 p 个元素、后面 n−p（个），上面是 O(n)，（后）整个反转也是 O(n)，**它的时间复杂度其实是 O(2n)，但是一般我们说时间复杂度是常数项忽略的，就是 O(n)**；空间复杂度是没有用空间的，就 O(1)。」（`[00:37:25]`–`[00:37:44]`；⚠ 转写把这里的 O(n) 记成了「O1」，按上下文的 O(2n) 与后一句单独的「空间 O(1)」可判定为 O(n)）

② 他推荐的通用解（开 p 个辅助数组）——「你最简单的做法肯定是把这些东西放到一个辅助数组，然后再把后面 4 个往前移，接下来再从辅助数组接回这个原数组的末尾。这种方法**虽然有空间复杂度**，空间复杂度是 **O(p)**，因为你要开一个 p 个大小的辅助数组，**但是它是比较好想的，而且比较万能，你换个其他题目也能去开数组。**」（`[00:37:58]`–`[00:38:23]`）他还专门点了链表：「就算是链表你都可以去开数组，先把链表存到数组然后再操作——这种做法往往都是比较万能的，**起码能保证考试你碰到一个算法题，你都是能写的，不至于空着。**」（`[00:38:28]`–`[00:38:42]`）

```
/* 数组 R[0..n-1] 循环左移 p 位（0 < p < n）；函数不需要返回值 */
void func(int R[], int n, int p) {
    int i;
    int *tmp = (int *)malloc(p * sizeof(int));  /* 开 p 个元素的辅助数组 */
    for (i = 0; i < p; i++)       /* 把 R 的前 p 个元素暂存到 tmp */
        tmp[i] = R[i];
    for (i = p; i < n; i++)       /* 把后面的 n-p 个元素整体往前移 p 位 */
        R[i - p] = R[i];
    for (i = 0; i < p; i++)       /* 把 tmp 里暂存的 p 个元素接回末尾 */
        R[n - p + i] = tmp[i];
}
/* 他讲过两种等价写法：第二个循环也可写成 for (i=0; i<n-p; i++) R[i]=R[i+p];
   第三个循环也可写成 for (i=n-p; i<n; i++) R[i]=tmp[i-(n-p)]。 */
```

③ 函数签名与命名。「这个函数其实并不需要返回值」「函数名你可以随便取，你懒得取名的直接 `FUNC` 就行，无所谓」「它的（已知）参数：一个是有一个数组，还有一个数组的大小 n，还有一个左移 p，这三个参数，就是把已知条件都当成函数参数传进来」「尽量这个起名字要和题干保持一致」（`[00:41:32]`–`[00:42:09]`）。

④ 复杂度。空间 **O(p)**、时间 **O(n+p)**（转写作「欧恩加匹」）：「时间的话你就看循环——前面有一个 O(p) 把这 p 个元素放到辅助数组，后面有一个 n−p 把后 n−p 个往前移，之后再把辅助数组的（元素）搬回来，对，加起来应该是 O(n+p)」（`[00:50:28]`–`[00:51:02]`）。

⑤ 他的收尾提醒。「初期学代码不要过多去纠结这些细节，初期学的话你只要把核心思路（掌握）——比如这个题目的核心思路就是开辅助数组；然后具体的这些写法……哪怕你和我写的再不一样也都行，**只要能把你的**

思路实现出来就行。第一遍学代码题就是学一下思路、然后稍微熟悉一下语法就够了。」（[00:56:02]–[00:56:27]）

4.2 非真题 数组反转（第 2 题）：左右指针

他说这两道非真题「也是要练习一下的：**一个是把数组反转，O(1) 的空间**；另外一个是说把两个有序的（序列）变成一个新的有序的」（[01:01:39]–[01:02:09]）。

> 「你现在有两个指针，一个指向左边、一个指向右边……因为现在要把数组反转，所以你每次把左右指针指向的位置对调……接下来把左指针往右走、右指针往左走，继续对调……**当两个指针都指到（中间）的时候，说明它在正中间，就不需要管了**，现在就反转了。」（[01:03:37]–[01:04:30]）

他现场演示的配对关系是：首尾 **7 和 6** 对调 → 第二组 **4 和 1** 对调 → 第三组 **5 和 8** 对调 → 中间剩 **3** 不动（[01:03:55]–[01:04:28]）。⚠ 转写只念出了这三组配对与「中间是 3」，没有完整念出该序列；按配对关系复原即为 `7 4 5 3 8 1 6`，但本资料不把它当作老师原话。

```
/* 数组反转：左右指针向中间靠拢
   ⚠ 老师本讲只讲了思路（「代码也可以写一下作为一个练习」[01:04:01]），
     没有逐行念代码 —— 下面这段是编者按他的思路补写的（非老师口径）。 */
void reverse(int a[], int n) {
    int i = 0, j = n - 1;
    while (i < j) {
        int t = a[i]; a[i] = a[j]; a[j] = t;   /* 交换左右指针指向的元素 */
        i++;  j--;                             /* 左指针右移、右指针左移 */
    }
}
```

4.3 顺带复习的快排一轮划分（同一套左右指针）

> 「快排注意它的思想应用：首先它就是**双指针**，所以双指针的它是**左右指针**；其实这个快排的话需要**从右往左开始**；然后它还有一个东西叫**枢轴**；第三个性质的话**每次发生交换、改变移动方向**。」（[01:05:28]–[01:05:58]，转写作「输走」）

他现场走了一遍：把枢轴 **6** 取出来放在外面（不要了），然后从右往左开始比——**比 6 大的放右边、比 6 小的放左边**。⚠ 他口播时先说反了一次（「比 6 大的都放在左边」），随即改口为「你比 6 大的都放在右边，比 6 小的都放在左边」（[01:06:35]–[01:06:43]），下表按**改口后的口径**整理。

动作	他的话
右指针的 **7** 比 6 大 → 不管，右指针左移	「这个 7 比 6 大，所以这个不用管」（`[01:06:48]`–`[01:06:54]`）
4 比 6 小 → 把 4 放到左指针停留的位置；**发生交换，改变移动方向**（从右往左 → 从左往右）	「4 比 6 小，所以小的应该放在左边，所以你会把 4 放在左指针停留的位置」（`[01:06:56]`–`[01:07:10]`）
从左往右：**1**、**5** 都比 6 小，继续走；**9** 比 6 大 → 把 9 放到右指针停留的位置 → 又交换，方向改回从右往左；**8** 比 6 大，不再移动	「9 比 6 大，大的应该放在右边，应该把 9 放在右指针停留的位置」（`[01:07:16]`–`[01:07:31]`）
两指针相遇 → 把 6 填进来，**左边都比 6 小、右边都比 6 大**	「你把 6 填进来，这时候你会发现左边都比 6 小、右边都比 6 大」（`[01:07:35]`–`[01:07:40]`）

⚠ 转写没有完整念出这个演示序列（只念到 7、4、1、5、9、8 与枢轴 6），本资料**不回推该序列**。他为什么要在这里插快排：「快排的话是选择题里面也是一个比较重要的点，而且刚才我们也说了，**快排那边是有一道代码题的大题的**，可以先把这个快排的流程再回忆一下」（`[01:07:43]`–`[01:07:55]`）；代码留到后面讲——「这个快排的代码不太适合一开始就讲……后期会讲代码」（`[01:05:23]`–`[01:05:28]`）。

4.4 2011 两个等长升序序列的中位数（第 11 题）：归并

题面要点：两个序列都已经有序、**等长**，求中位数。他特别强调定义的细节：「中位数人家规定注意，必须是**二分之（L）还要向上取整**，人家规定了向上取整。之前可能学（数学）中位数是（偶数个取）中间两个求一下平均这种的，现在人家规定就取这个向上取整，比较按题意来。」（`[01:00:24]`–`[01:00:39]`）他给的例子是「上边五个元素、下边五个元素，加起来是十个元素，那十个元素的中位数就是第五个」，合并后的前五小是 **2、4、6、8、11**，所以中位数是 **11**（`[01:00:41]`–`[01:01:00]`）。教材补充 他与别的题目里用的数字对得上 2011 年真题题面自带的例子（两个序列为 11,13,15,17,19 与 2,4,6,8,20，答案 11），但**他本讲只念了合并后的前五个数，没有念出这两个序列。**

① 他推荐的通用解（归并到辅助数组）。「其实这个题目不开辅助空间也行……因为你的目的就是求出第五小的，就是中位数。你第一种方法是把它们合并到一块，把这个完整的序列求出来，你想要求第五个，你找下标为四的就可以了；另外一种做法的话其实你都不用辅助数组，你只要去统计……**但是这个题目还是要用一下辅助数组**，为什么？就是如果你后面学习到归并排序的代码，这个是有用的，所以这边还是要写一下。」（`[01:15:04]`–`[01:16:12]`）

```
/* 两个等长升序序列 S1、S2 各长 L，返回它们的中位数 */
int func(int S1[], int S2[], int L) {
    int i = 0, j = 0, k = 0;
    int *tmp = (int *)malloc(2 * L * sizeof(int));  /* 开 2L 个元素 */
    while (i < L && j < L) {            /* 谁小取谁，取完指针往后走 */
        if (S1[i] < S2[j])  tmp[k++] = S1[i++];
        else                tmp[k++] = S2[j++];
    }
    while (i < L)  tmp[k++] = S1[i++];  /* 第一个序列还有剩，全部接后面 */
    while (j < L)  tmp[k++] = S2[j++];  /* 第二个序列还有剩，全部接后面 */
    return tmp[L - 1];                  /* 中位数下标是 L-1，这种细节一定要注意 */
}
/* 他还说两个序列不等长也能用：「如果这两个数组的长度分别是 m 和 n，
   你把 L 换成 m 和 n 就行了」[01:17:57]–[01:18:02] */
```

② 他的简化解（不开辅助数组，只数到第 L 个）。「你只要去统计……比如说 2 和 11 相比 2 更小，那第一个小的是 2；那之后 4 和 11 相比，第二小的是 4；你只要这样去数，**数到第五小的那就是中位数**，其实都用不到这个辅助数组。」（`[01:15:36]`–`[01:15:54]`）

```
/* 不开辅助数组：数到第 L 个就是中位数 */
int func(int S1[], int S2[], int L) {
    int i = 0, j = 0, cnt = 0, ans = 0;
    while (i < L && j < L) {
        ans = (S1[i] < S2[j]) ? S1[i++] : S2[j++];  /* 三目运算符，一步到位 */
        if (++cnt == L) return ans;                 /* 第 L 小就是中位数 */
    }
    return -1;  /* 循环内一定返回，这句只为语法规范，不会执行到 */
}
```

他点名的三处细节

① 循环里一定会返回，最后也必须写 return。「即便你知道你的代码在 while 循环内一定能返回，但是按照语法上，因为返回（值）是 int，你这里一定要有 return。比方说你可以写一个 `return -1`，然后解释一下：这里一定不会执行，在循环内部的 return 一定会找到中位数。」（`[01:30:55]`–`[01:31:26]`）

② 不用处理「一个走完另一个没走完」。「为什么下面没有加这些 while 循环？这个不需要考虑的，因为你保证这个 L 是一定能找到中位数的，不用再去处理这个没比完的情况了。」（`[01:29:12]`–`[01:29:51]`）

③ 三目运算符。「`ans = (S1[i] < S2[j]) ? S1[i++] : S2[j++]`——条件成立就取前面的、不成立就取后面的，**整个 if 都可以不要了**，这种语法平常使用还是挺高频的。」（`[01:32:11]`–`[01:33:57]`）

③ 复杂度。归并版：「它的空间是 O(2L)，因为开了 2L 大的数组；时间的话也是 O(L)」（`[01:23:10]`–`[01:23:23]`）。简化版：「空间复杂度就是 O(1)，没有用空间了，就定义了几个变量；时间复杂度就是 O(L)……像刚才要走 2L 次把数据全部处理完，现在你处理到 L 次就行了」（`[01:29:56]`–`[01:30:17]`）。

4.5 非真题 两个有序序列归并（第 6 题）

他先讲了归并的完整规则，再去看第 11 题的代码：「谁小的话把谁取出来……谁小的话你就把谁的指针往后走……**当一个序列走完之后，那你把剩下这个序列的全部接在后面就可以了**，这样就整个有序了。」

（[01:11:48]–[01:12:32]）他现场报出的比较序列是 1 比 2（取 1）→ 2 比 6（取 2）→ 5 比 6（取 5）→ 6 比 9（取 6）→ 8 比 9（取 8）→ 9 比 12（取 9）→ 10 比 12（取 10）……⚠ 转写只念到这些数、没有完整念出两个序列本身（他在 [01:11:07]–[01:11:20] 单独念到过 1、6、8、12、15、18），**本资料不回推两个序列**。

他点明这道题和第 11 题的关系：「这个第 6 题是（第 11）题的一个子类，这个代码是特别相似的」（[01:12:57]–[01:13:07]）——也正是他为什么先讲归并、再讲中位数。他还交代：「这种代码可能书上都没有、王道书的课后答案都没有，所以这边写一下，到时候课下你可以对着上课写的自己动手写一写」（[01:18:56]–[01:19:06]）。

4.6 2013 主元素（第 12 题）：辅助数组当哈希

题面要点：他说这道题的题面里**范围是给了的**（「首先原数组的范围给了，因为没有给你就没法用辅助数组这个方法」），$0 \le a_i \le n-1$；出现次数**大于 n/2** 的元素叫**主元素**（转写作「组元数」）；而且「这个题目要注意，题目里人家说**否则输出 -1**」（[02:04:26]–[02:05:26]，转写作「否则辅助速度负一」）。

> **他说这道题「有输入输出你还是要写的」——这是三段式唯一被打破的地方**
>
> 「这个题目虽然平时的话不需要输入输出，但是这个题干有输入输出**你还是要写的**，你不能最后返回 -1 就不管了，要把它输出的」；「这个函数是**并不需要返回值**的，因为这里是**输出** -1 并不是返回 -1」（[02:05:26]–[02:05:51]）。⚠ 这与 §3.1 里「输入输出都不要写」的通则不矛盾——**题干明确要求输出时才写**。

```
/* 求主元素（出现次数 > n/2 的元素）；有则输出主元素，否则输出 -1 */
void func(int A[], int n) {
    int i;
    int *temp = (int *)malloc(n * sizeof(int)); /* A[i] 最大取到 n-1，开 n 个够 */
    for (i = 0; i < n; i++) temp[i] = 0;        /* 手动清零，别省这一步 */
    for (i = 0; i < n; i++) {
        temp[A[i]]++;                   /* 把 A 的值当辅助数组下标统计次数 */
        if (temp[A[i]] > n / 2) {       /* 每统计一次就判断，超了就是主元素 */
            printf("%d", A[i]);         /* 输出的是主元素本身，不是 temp[i] */
            return;                     /* 必须 return，不能写 break */
        }
    }
    printf("-1");                       /* 走完都没超，输出 -1 */
}
```

他给的另一种写法：先只统计，统计完再单独写一个循环遍历辅助数组判断——「前面单独统计次数，统计完次数之后单独再写个循环去判断」（[02:15:41]–[02:15:57]）。两种写法的时间复杂度都是 O(n)：「你其实循环执行了两次——写在上面（的话）循环就执行一次，写在下面两次；但是你无论一次两次，它都是 O(n)」（[02:18:18]–[02:18:27]）；空间 O(n)。

他逐条点出来的四个易错点（都是判卷会扣分的）

① **找到主元素后必须 return，不能 break。**「找到之后你就可以直接写一个 break 或者 return——**不能加 break。**可以想一下：如果你 break，只是退出循环了，（后面那句输出 -1）一定会执行；**一定不能加 break，必须加 return。**」（`[02:16:17]`–`[02:16:43]`）

② **输出的是主元素，不是它出现的次数。**「这里千万写上（`i` / `A[i]`），不要写上 `temp[i]`——这里是刚学的话容易搞混：`temp[i]` 是它出现的次数，你现在要输出的肯定是主元素。」（`[02:14:57]`–`[02:15:16]`）

③ **n/2 要写除法，不能写分数线。**「你写的这个二分之（n）其实一般初学会这样写，真正（写）代码你敲代码肯定是要 `n / 2` 的，没有这个分数线的……虽然这个东西不是大问题，但是对于判卷老师，他会看起来你写的稍微专业一点，留个印象分。」（`[02:08:50]`–`[02:09:15]`）

④ **细节问题只扣一两分，别因小失大。**「这些细节问题考试的话可能扣个一两分，你只要把最核心的思路、代码写个差不多，大体分数就能拿到了，剩下的细节问题稍微检查一下。」（`[02:16:50]`–`[02:17:02]`）

他顺带交代了这道题答案的替代做法：「十二题它有一个东西专门有这样的一个算法叫**摩尔投票（法）**，但是这个就可以不看了」，「十二题的答案也不推荐看了，它也是不具有通用性，都是最优解」（`[01:52:03]`–`[01:52:21]`）。

4.7 2018 未出现的最小正整数（第 13 题）：辅助数组的三种开法

题面要点：求**未出现的最小正整数**；原数组**有负数**（他给了两个用例：第一个数组答案是 1、第二个数组答案是 4）。他给出了本题唯一的推理：「这个未出现的最小正整数，它最大一定是（n+1）……因为原来的话有三个数，你让它多连续着——1、2、3 都有，那这时候没出现的就是 4」（`[01:33:34]`–`[01:33:44]`；`[02:23:34]`–`[02:23:56]`）。

本题的核心：辅助数组到底开多大（他单独拎出来讲了三遍）

「这个题目开 **N 个**是最麻烦的，因为那个下标要减 1 去放；你开 **N+1** 跟 **N+2** 其实都还好。」（`[02:46:34]`–`[02:46:40]`）「这个题目其实（最优解）开 **N+2 个**是最好的……写起来应该是绝对是最好理解的，但是答案是开那个（N 个）。」（`[02:36:42]`–`[02:36:54]`）

空间上不吃亏：「注意这个空间不算额外开销，N 和 N+2 没多大（区别）……判分的时候不会因为你多开（两）个空间去扣分的，它都是 O(n)。」（`[02:36:54]`–`[02:37:03]`）

```
/* 写法一（答案的写法）：辅助数组开 n 个，存的时候把值减 1 —— 最容易写乱
   0 号下标用来放「1」，下标 i 代表数值 i+1 */
int func(int A[], int n) {
    int i;                                  /* i 要定义在循环外面，后面要用到 */
    int *temp = (int *)malloc(n * sizeof(int));
    for (i = 0; i < n; i++) temp[i] = 0;
    for (i = 0; i < n; i++)
        if (A[i] >= 1 && A[i] <= n) temp[A[i] - 1]++;   /* 减 1 存 */
    for (i = 0; i < n; i++)
        if (temp[i] == 0) break;            /* 找到第一个 0 就停 */
    return i + 1;   /* 取的时候要加回来；1..n 都出现时 i=n，返回 n+1 */
}
```

```
/* 写法二（他推荐）：辅助数组开 n+2 个，原封不动存 —— 最好理解、最不容易错 */
int func(int A[], int n) {
    int i;
    int *temp = (int *)malloc((n + 2) * sizeof(int));
    for (i = 0; i < n + 2; i++) temp[i] = 0;
    for (i = 0; i < n; i++)
        if (A[i] >= 1 && A[i] <= n + 1) temp[A[i]]++;   /* 原封不动存，不减 1 */
    for (i = 1; i <= n + 1; i++)            /* 从下标 1 开始找第一个 0 */
        if (temp[i] == 0) return i;
    return -1;      /* 一定不会执行到，只是为了保证有返回值（编译通过） */
}
```

为什么要加 `A[i] >= 1` 和上界这两个判断——他给的例子：「你比如说我现在给你一个用例，总共就三个数，它是 6、5、100……那你没出现的最小正整数肯定就是 1；但是你如果只开一个三个大小的数组，你这个 100 是没法作为下标放进去的。因为这个题目**没有给你这个原数组的数值范围**。但是这个 100 就没必要了、没有用——因为你三个数、一共就三个元素，未出现的最小正整数最大只能是 4（= n+1），所以你高于 4 的没有意义，就不管它。」（`[02:27:43]`–`[02:28:37]`）

要不要写 `++`：「这个题目只用来统计是否出现过，不需要（次数），其实你可以强行赋值 1……但是我觉得其实你还是按照加加更好，因为（那样）适用范围更广：既能统计次数，然后（次数）为零就是没出现。」（`[02:37:17]`–`[02:38:03]`）

写法三（N+1）的那个坑，他专门警告过：「如果你只开 N+1 个……前面都没问题，但是你这个（循环）条件最后还要加上一个：返回 N+1——这里就不是 `return -1` 了，你要 `return` 一个 N+1。很有可能出现（1、2、3 都有）这种情况，那你就不会（在循环里）返回，最后还得额外处理。如果用 N+1，**这一句很有可能被忽略掉，就是一个扣分的点**；但如果你开 N+2，就算没写 `return -1` 照样没问题。」（`[02:43:11]`–`[02:44:58]`）

4.8 拓展：C++ 自带的 map（他定位成「压箱底的」）

> 「（用辅助数组）它很局限：首先（原）数组 a_i 必须是正整数……但是有时候哪怕是负数你也要统计个数的，像这种情况它就没法处理了；必须（是）整数的话你得提前知道它的范围，这个就很麻烦。」（`[02:48:05]`–`[02:48:57]`）

> 「这个当然考试的话，你如果能用（辅助数组就）尽量用这种，因为这是**最标准的解法**。你如果用（编程语言）自带的工具，就相当于你用了那种专门的库函数，**可能会扣一点点分**；但如果那个题目你这种方法不能做，你相当于就要空着了，但是你用那个编程语言自带的工具，它依然是可以写的。」（`[02:49:15]`–`[02:49:51]`）

```
/* C++ 自带 map（本质是哈希表）：键可以是任意类型（含负数、字符）
   原数组里有几个不同的值，就开几个空间，提前不用关心取值范围 */
map<int, int> temp;                    /* 第一个参数是键，第二个参数是值 */
for (i = 0; i < n; i++) temp[A[i]]++; /* 写法完全仿造数组，但 A[i] 是「键」不是下标 */

/* 遍历只能用迭代器（不能用下标）：auto 是 C++ 自带的，因为类型名特别长 */
for (auto it = temp.begin(); it != temp.end(); it++)
    printf("%d %d\n", it->first, it->second);   /* ->first 是键、->second 是值 */
/* 例：字符数组 C A C B A 统计完，C 出现 2 次、A 出现 2 次、B 出现 1 次 */
```

他给的三点使用提示：「**这个 Map 其实就是哈希表**，就是（编程）语言自带的哈希表，它有空间开销，但是时间开销很小，**查找的话 O(1)**」（`[03:03:13]`–`[03:03:31]`）；「**真题里面从来没有考过**，作为拓展了解一下」（`[02:56:48]`）；「一定要作为**最后压箱底的**再用，普通方法解决不了再用这个」（`[02:59:00]`–`[02:59:07]`）。

4.9 2020 三数组最短距离与三元组（第 14 题）：暴力解收官

> 「这是 20 年的一个**真题**。这个题目的最优解有时候不太好想，还得去推导，所以很多人复习了很久，算法题最后（考试）还是用了暴力解。」（`[03:05:42]`–`[03:05:55]`）

题面要点（他念的）：「它有三个数组，三个数组**不等长**、长度都不一样。然后你要从每个数组里面取出一个元素，分别是 a、b、c；然后你把任意两者之间求一下**绝对值**，然后让你求**最短距离**，他还问了相应的**三元组**。」（`[03:05:55]`–`[03:06:21]`）

```
/* 三个数组 S1、S2、S3，长度分别为 n1、n2、n3；返回最小距离 */
int func(int S1[], int S2[], int S3[], int n1, int n2, int n3) {
    int i, j, k;
    int ans = INT_MAX;              /* 求最短距离必须初始化成最大值 */
    int x = 0, y = 0, z = 0;        /* 用来存最小的那个三元组 */
    for (i = 0; i < n1; i++)
      for (j = 0; j < n2; j++)
        for (k = 0; k < n3; k++) {
            int d = abs(S1[i] - S2[j]) + abs(S2[j] - S3[k]) + abs(S1[i] - S3[k]);
            if (d < ans) { ans = d; x = S1[i]; y = S2[j]; z = S3[k]; }
        }
    return ans;
}
/* 他点名的编程技巧 + 可直接用的函数：
   ① 「你要求最短距离你不能初始化成 0，因为你初始化成 0，别的距离都比 0
      大，就不会更新了。所以一般求最短距离，你这个初始值都是最大值——
      INT_MAX 表示 int 的最大值，这个可以直接用。」[03:09:16]–[03:09:46]
   ② 「绝对值函数都可以直接用，你直接 abs 求绝对值，因为这种你单独再去
      实现一下没有太大意义……它在这个题目里面只是非核心的一小步。」[03:12:01]–[03:12:17] */
```

他还带了一句 int 最大值怎么来的（与（计）组的知识点打通）：「其实它的本质上……如果 int 是 32 位，32 位你想要求最大值，就是**符号位为 0、后面 31 位全 1**，写成十六进制就是这样的，`0x` 表示十六进制。答案是把这个常数定义成了一个变量，**没必要的**——这个东西默认它就是这么多，编程语言里面帮你定义好了，不需要自己再显式地去写，但是知道这个流程就可以。」（`[03:09:51]`–`[03:10:34]`）⚠️ 他口播时先说「后面 32 位全 1」、随即改口为「后面 **31** 位全 1」，转写里**没有念出具体的十六进制常数**；教材补充 按他给的定义写出来就是 `0x7FFFFFFF`，但这属于教材补充、非老师口径。

复杂度与他给的得分判断：「空间复杂度是 O(1)，时间复杂度是 O(n³)……因为它没给（长度），那你就当作每一个数组都平均是 N。只要整个（作答）体系是完整的，无论你的开销多大，只要一二三问都回答了、代码也都实现了，最后 15 分你这个保底肯定是有这个 11 分左右。」（`[03:15:33]`–`[03:16:11]`）

他也把「为什么不值得死磕最优解」说透了：「毕竟代码题才占 15 分，大题的话还有这个 55 分都是非算法，而且四本书还有很多的选择题……因为算法题如果说你花时间太多，很有可能考的还不是你学的那个，所以这个性价比特别低。而且这些方法（答案的最优解）确实不具有通用性。」（`[03:14:20]`–`[03:15:02]`）

5 真题年份对照与认定依据

课上他是**按书上的题号**讲题的（他念的是「第十题、十一题、12、13、14 题」，另加第 2、6 题两道非真题），**五道真题里只有第 14 题他亲口说了年份**。「2.2 节」这个归属来自他说的「在书上的 2.2，就是顺序表，这边是有 5 道」，与这 5 题一一对应（⚠️ 他没有逐题说「这是 2.2 的第 N 题」，这是编者的对应）。下表把年份与题号对上，并写明依据——凡不是他本讲说出来的，都在「依据」列标出来源。

题号	主题	年份	认定依据
第 14 题	三数组最短距离与三元组	**2020**	✅ **老师本讲亲口说出**：「这是 20 年的一个真题」（`[03:05:42]`）
第 10 题	循环左移 p 个位置	2010	⚠️ 他**没有说年份**。依据：题目特征（「0 < p < n 的循环左移」）与 2010 年 408 数据结构第 42 题一致（CSDN、2010 年 408 真题）
第 11 题	两个等长升序序列的中位数	2011	⚠️ 他**没有说年份**。依据：题面定义（⌈L/2⌉ 向上取整）与 2011 年 408 数据结构第 42 题一致（2011 年 408 真题）
第 12 题	主元素（出现次数 > n/2）	2013	⚠️ 他**没有说年份**。依据：题面「$0 \le a_i < n$、$m > n/2$」与 2013 年 408 数据结构第 41 题一致（2013 年 408 真题）
第 13 题	未出现的最小正整数	2018	⚠️ 他**没有说年份**。依据：题面「含 n 个整数的数组、找未出现的最小正整数」与 2018 年 408 数据结构第 41 题一致（CSDN）

两处交叉验证（编者加，非老师口径）：① 王道书按年份升序排题号，第 14 题 = 2020 年，「10→11→12→13」正对应 2010→2011→2013→2018，与逐题特征自洽；② 他说 2.2 节「一共有 5 道」代码题，与上表 5 道一一对应。**这两条只是佐证，年份的第一手依据仍是上表的题目特征比对。**

6 易错点清单（他警告过的，按他点名的顺序排）

#	错误	他的原话 / 出处
1	**只写思路、不写代码**	「你最后代码什么解法都没写，空着了只写一个思路，那可能给的分就很少」（`[00:27:07]`）
2	**在答卷里写头文件 / main / 输入输出**	「头文件……和 main 函数，包括输入输出这些都不要写」（`[00:24:10]`） ⚠️ 例外：题干明确要求输出时（第 12 题）必须写
3	动态数组写成 `int a[p]`	「这个不太推荐……这种写法一般编译器不会通过」（`[00:42:22]`–`[00:44:47]`）
4	用「数组下标多一个少一个」的错误收场	「一定要通过一个具体的例子去代一下，看你的代码能不能正常运行」（`[00:55:48]`）
5	开完辅助数组忘记手动清零	「有的内存里面可能有一些旧的数据，所以你最好手动……先赋成零」（`[02:07:05]`）
6	循环里一定能返回，就省略最后的 `return`	「因为返回（值）是 int，你这里一定要有 return」（`[01:31:10]`）
7	代码里写分数线 `n/2`	「敲代码肯定是要 `n / 2` 的，没有这个分数线的」（`[02:08:53]`）

#	错误	他的原话 / 出处
8	**输出主元素时写成 `temp[i]`**	「`temp[i]` 是它出现的次数，你现在要输出的肯定是主元素」（`[02:15:02]`）
9	**找到主元素后用 break 而不是 return**	「你 break 只是退出循环了，（后面那句）一定会执行……必须加 return」（`[02:16:31]`–`[02:16:43]`）
10	辅助数组开 n 个时「减 1 存、忘了加回来」	「放的时候下标减 1 了，拿的时候又忘了加……很容易漏掉」（`[02:46:51]`）
11	**开 n+1 个时漏掉「返回 n+1」那一支**	「很有可能会出现（1、2、3 都有）这种情况，那你就不会（在循环里）返回……很有可能这个被忽略掉，就是一个扣分的点」（`[02:43:11]`）
12	求最短距离把 `ans` 初始化成 0	「别的距离都比 0 大，就不会更新了……初始值都是最大值」（`[03:09:19]`）
13	注释乱加或干脆不加	「不要乱加，加的一定要简洁，加在该加的位置」（`[02:13:00]`）
14	照抄答案的最优解	「答案给的都是最优解……换个题目就没法用了」（`[03:07:16]`）

7 考场速查卡

作答格式（照做）

三段式：①中文写思路（有分，不会写代码也要写）→ ②写核心函数 → ③写时空复杂度（直接给结果，最好补一句原因）。

只写一个核心函数，已知条件全当参数；**不写头文件、main、输入输出**（题干要求输出除外）；**加注释**，代码控制在 10~15 行；`swap`、`abs`、取最大最小直接用；**答案写在显眼位置**。先写代码、再回头写第一问的思路。

场景	套路	复杂度
循环左移 p 位	开 p 个辅助数组暂存前 p 个 → 后面 n−p 个前移 → tmp 接回末尾 （答案的最优解是三反转，他不推荐）	时间 O(n) 空间 O(p)
数组反转	左右指针向中间靠拢，边靠边交换，相遇即停	空间 O(1)
两有序序列归并	谁小取谁、指针后移；一个走完就把另一个剩下的全接后面	时间 O(n)
两等长升序序列的中位数	① 归并到辅助数组，返回 `tmp[L-1]`；② 不开数组，用 `cnt` 数到第 L 个就返回	①O(L)/O(L) ②O(L)/O(1)
主元素（次数 > n/2）	开 n 个辅助数组清零 → `temp[A[i]]++` → 每次加完就判断，超了就输出 `A[i]` 并 return	时间 O(n) 空间 O(n)
未出现的最小正整数	**辅助数组开 n+2 个**：原封不动存（只收 1 ~ n+1）→ 从下标 1 找第一个 0 → 返回下标 （开 n 个要减 1 存，最容易写乱；开 n+1 个要额外处理返回 n+1）	时间 O(n) 空间 O(n)
三数组最短距离	三重循环枚举；`ans` 初始化成 `INT_MAX`；`abs` 直接用；顺手存三元组	时间 O(n³) 空间 O(1)

三条开数组纪律（背）

① `int *t = (int *)malloc(k * sizeof(int));`（C++ 也可 `new int[k]`），不要 `int t[k]`；

② 开完**手动清零**；

③ 大小**慎重定**，定了之后「**前后都要对应**」——存的时候怎么偏移，取的时候就要怎么还原。

三句提醒（他最常重复的）

「**一定不要空着**」；「**只要整个体系三问都回答了、代码也都实现了，保底 11 分左右**」；「**不要硬背代码，多写几遍**——考前写四五遍，语法自然就熟了」。

附：本资料的来源与边界

① 素材与转写方式。只依据一份 ASR 转写稿：`p02_...顺序表算法题.md`（2548 行，带 `[HH:MM:SS]` 时间戳）与其同内容 `.txt`；讲稿简繁混杂、同音错字密集（见文首「ASR 校正说明」）。**正文所有引语都是我在转写稿上做过错字校正后的文字，并在必要处注明校正了什么**；未校正的原始转写不作为引语。凡标注 `[HH:MM:SS]` 的引语都可按该时间戳回稿复核。

② 时长与时间轴。标称时长 **3 小时 21 分 25 秒**。转写稿首条时间戳是 `[00:08:50]`（此前 8 分 50 秒无语音，应为课前等待），末条是 `[03:18:21]`；中间 `[01:37:37]`–`[01:50:40]` 是课间休息（约 13 分钟无语音，他在 `[00:58:29]` 预告过「中间休息一个十到十五分钟」）。**本资料引用的时间戳全部照抄转写稿，未做位移。**

③ 录制时间（推断）。讲稿没写日期。依据「现在已经 9 月 5 号了」（`[00:10:11]`）与「（王道书）从 09 年到 23 年的真题都有，不包含 24 年真题」（`[00:16:15]`），可推知录制于 **2024 年 9 月 5 日**——⚠ 编者推断，非他所说。同理他讲顺序表热度时的「去年考的图、前年考的数、再前年考的图」**没点明年份**，本资料不代他补成 2023 / 2022 / 2021。

④ 转写含糊、本资料明确不猜的地方：（a）「最烈考了一道」（`[00:19:28]`）——在数非代码题分布时出现，与王道书章节名对不上，**未采信、未计入 §1.2**；（b）「09 到那是 15 年的整体」（`[00:16:24]`）——按「总共 15 道」自洽，判为「09 到 23 是 15 年的真题」，已在正文标注；（c）「挺堂之间没考了」（`[00:57:04]`）——判为「挺长时间没考了」，已标注；（d）循环左移最优解的时间复杂度——他口播「其实是 O(2n)，常数项忽略掉，就是 O（n）」，**转写把后者记成了「O1」**，按 O(2n) 与「空间是 O(1)」并存可判定为 O(n)，正文已标注；（e）int 最大值的十六进制常数——他只说「符号位为 0、后面 31 位全 1，`0x` 表示十六进制」，**没念出常数本身**，正文把 `0x7FFFFFFF` 标为「教材补充，非老师口径」；（f）**三处演示序列**（快排那一轮的初始序列、数组反转的初始序列、第 6 题的两个有序序列）转写都只念出部分数字，正文只写他念到的配对 / 比较关系，原处标注「不回推序列」；（g）第 11 题那两个序列——他念的「第五小是 11」与 2011 年真题题面自带的例子一致，但**他没念出序列本身**，正文标为「教材补充」。

⑤ 教材补充（非老师口径）的完整清单。除上面已标注的以外，还有：§4.2 数组反转的代码（他只讲了思路）、§4.9 里 `0x7FFFFFFF` 的写法、§5 表格里四个年份的比对结论与两处交叉验证。**其余全部内容（含所有代码骨架、复杂度、答题规范）都有讲稿出处。**

⑥ 本讲没有覆盖的内容：链表（2.3 节，4 道算法题）——他明确留到下一讲（「等到下次课的话，我们再讲一讲（链表）的四道算法题」`[03:16:25]`），双指针里的**快慢指针**也只点了用途；**快排的代码**——「这个快排的代码不太适合一开始就讲……后期会讲代码」（`[01:05:23]`），本讲只讲了一轮划分；**2.2 节剩下的题目**，含他提到的「有一道是给了代码让你去分析」（`[00:43:29]`，说「之后会讲」）；**非代码题的 15 道**只报了小节名与数量，没有逐题讲，数据结构以外的三科只各提了一句。

⑦ 篇幅取舍。本讲 3 小时 21 分里，前 40 分钟是考情与规划、后 2 小时 30 分集中在 2.2 节这 5 道题上，因此本资料把考情压成一节、五道题按「题面要点 → 他的通用解 → 复杂度 → 他点名的陷阱」各压缩到一页左右，**删掉的是重复口播与教材通识，没有删考点、代码与真题解答**；第 13 题的三种开法、第 12 题的四个易错点这类他反复强调的部分予以保留。

本资料为《408 各科大题知识点强化课》第 1 讲（分 P2「数据结构顺序表算法题」）的自学整理稿，全部内容以该讲 ASR 转写稿为准，未使用课件原文。转写稿的错字已按文首说明校正；凡标注「教材补充，非老师口径」「⚠」的地方，请以王道书与历年真题原文为准。文中出现的年份，除 2020 年由老师亲口说出外，均为编者按题目特征比对历年真题后的标注（依据见 §5），**不代表老师本讲的说法**。